

AI 使用说明情况

第二次挑战

本次挑战中，我使用 AI 辅助完成环境检查、前缀缓存实验脚本编写、实验结果分析和请求流程学习。以下说明记录实际使用方式，并区分 AI 提供的内容与我完成的操作和理解。

一 使用的 AI 模型

使用工具：Codex 中的 AI 助手。具体模型版本：GPT-6 sol。

二 使用的代表性提示词

我通过多轮对话提出问题，并提供终端输出、实验结果和源码文件。以下按实际讨论内容归纳代表性提示词，表述经过整理，并非逐字记录。

1. 结合我第一次挑战的完成情况，评估第二次挑战的难点、所需基础和执行顺序。
2. 看一下我这个输出，没问题的话提供一下后续启动与实验步骤。
3. 帮我检查我这个实验输出，看一下 token 计数、缓存字段及指标是否符合任务要求。
4. 给我解释一下请求队列、Prefill、Decode、KV Cache 与流式返回，这些概念我不是很难理解，用简单例子解释一下。
5. 我对于前缀复用和请求流程的理解的表述正确吗，有没有什么问题，还有为啥前面的 token 不同时为何会影响后续 KV。
6. 帮我找一下关键源码的位置，我要如何阅读这个源码文件，给我提供一些建议。
7. 根据我提供的源码，总结一下请求入队、前缀匹配、批次构建和结果处理的关系。

三 AI 提供的帮助

环境与实验方面，AI 根据我提供的终端输出确认原有 GPU、模型和虚拟环境仍可使用，指导补装 Ray 2.56.0、恢复 CUDA 路径并启动服务。AI 编写了 `prefix_benchmark.py`，实现两组各 32 条请求、最大并发 8、清缓存、预热、流式接收和指标统计。我将脚本上传到云端运行，再把实际输出交给 AI 检查。两组均成功完成，数据来自真实实验运行。

学习方面，AI 帮助提取和定位关键源码，并解释请求排队、前缀匹配、Prefill、Decode、KV Cache 和流式返回。我最初只看函数名仍不理解，随后要求从基本概念重新讲解，并用自己的话回答问题，让 AI 纠正不准确的表述。AI 还提供源码阅读顺序和报告说明草稿。实验脚本的主要代码和部分说明文字由 AI 提供，我如实说明这部分帮助。

四 AI 回答的局限与核对过程

我没有把 AI 的解释直接当作已经验证的事实。环境状态通过版本输出和服务日志确认；实验结果通过逐请求记录、流式元数据和汇总文件核对。例如，两组均为 32 条成功请求，每条输入 2112 个 token、输出 16 个 token；共享组每条命中 2048 个 token，剩余 64 个需要 Prefill。这些记录支持缓存减少重复输入计算的结论。

使用过程中，AI 最初从函数名和调用链讲解，默认我已经理解推理系统的基础概念。我虽然看到了说明，仍不明白这些组件是什么。后来通过简化例子和问答，才逐步理解。

源码定位也存在不够准确的地方。最初生成的索引包含大量其他模型和后端的 forward 函数，材料过长；后续才筛选出本次普通生成请求需要的函数。AI 提供的源码路径位于云端隐藏的 .venv 目录，我无法直接在本机找到，后来通过复制到普通目录、下载源码文件解决。

五 对 AI 生成内容的使用说明

本次 AI 参与了脚本编写、运行步骤指导、指标解释、源码学习、报告素材整理。本说明也由 AI 根据实际对话协助起草。我负责在云端执行操作、提供日志和文件，并通过问答表达和修正自己的理解；我不将 AI 编写的代码或文字全部表述为独立完成。